A Benchmarking Framework for Grid-Based Motion Planning: BFS, Dijkstra, A*, Theta*, and Jump Point Search

Preliminary Evaluation in ROS 2 with Python: Runtime, Path Optimality, and Memory Analysis

Mohamed Eljahmi
Robotics Engineering Department
Worcester Polytechnic Institute
100 Institute Road
Worcester, Massachusetts 01609

***Abstract*— This project began with a narrow focus: implementing and benchmarking five classical grid-based search algorithms—Breadth-First Search (BFS), Dijkstra's algorithm, A*, Theta*, and Jump Point Search (JPS). These methods remain the immediate scope, providing a well-understood testbed where trade-offs in runtime, path length, and node expansions can be compared. The framework is implemented in ROS 2 with Python, with utilities for random obstacle generation, ASCII map loading, start/goal specification, and reproducible benchmarking. At this stage, BFS and A* are integrated, with benchmarks contrasting uniform-cost exploration against heuristic-guided search under both Manhattan and Euclidean distance heuristics. These comparisons highlight how cost models and heuristic selection affect efficiency and path quality. While the foundation is grid-based, the architecture is deliberately extensible: its unified planner interface and benchmarking harness could support future extensions to incremental graph searches for dynamic maps, and even policy-based algorithms from reinforcement learning.**

***Keywords*— Grid-based motion planning, Breadth-First Search (BFS), Dijkstra's algorithm, A* algorithm, Manhattan heuristic, Euclidean heuristic, Theta*, Jump Point Search (JPS), incremental search, policy-based planning, reinforcement learning, benchmarking framework, ROS 2.**

I. Introduction

Path planning is a fundamental problem in robotics and artificial intelligence. Mobile robots, manipulators, and autonomous agents must be able to navigate from an initial configuration to a goal while avoiding obstacles. Among the many approaches to path planning, grid-based search provides a simple yet powerful model: the workspace is discretized into cells that are either free or occupied, and algorithms explore this graph to connect the start and goal states.

Classical grid-based planners such as Breadth-First Search (BFS), Dijkstra's algorithm, and A* are widely studied because they guarantee optimal solutions under appropriate cost models. BFS explores in uniform steps without considering costs, while Dijkstra extends this to weighted graphs by minimizing cumulative cost. A* further introduces heuristics, and its performance depends strongly on the heuristic function chosen—for example, Manhattan distance is consistent with 4-connected grids, while Euclidean distance matches 8-connected grids where diagonal moves cost √2. More advanced methods such as Theta* and Jump Point Search (JPS) improve efficiency by reducing node expansions or enabling any-angle paths. Each algorithm exhibits trade-offs in runtime, memory usage, and path quality, making systematic evaluation important for both education and practical deployment.

This project began with a deliberately narrow focus: to implement and benchmark these five classical grid-based planners in a consistent setting. Implemented in ROS 2 with Python, the framework provides random obstacle generation, ASCII map loading, start/goal specification, and benchmarking utilities. Algorithms can be executed under identical conditions, and metrics such as path length, number of turns, nodes expanded, runtime, and memory consumption are recorded.

By October 6, the framework will include initial implementations of BFS and A*, enabling preliminary results on random and structured maps. Dijkstra, Theta*, and JPS will be added in subsequent iterations. While the near-term scope is restricted to grid-based graph searches, the architecture is flexible and extensible: the same benchmarking structure could, in future work, support incremental planners for dynamic maps or even policy-based approaches from reinforcement learning.

II. Background

Grid-based path planning treats the environment as a discrete occupancy grid where each free cell corresponds to a vertex in a graph and edges connect neighboring cells. Search algorithms then explore this graph to connect a start cell and a goal cell. Several classical algorithms are widely studied:

A. Breadth-First Search (BFS)

BFS explores the grid in layers outward from the start, guaranteeing the shortest path in terms of number of steps when all moves have equal cost. Its drawback is high memory usage, since all frontier nodes are stored.

B. *Dijkstra's Algorithm*

Dijkstra extends BFS to weighted graphs by maintaining a priority queue over cumulative path cost. It guarantees an optimal path in graphs with positive edge weights but tends to expand many nodes because it does not use heuristics.

C. *A*Algorithm*

A* improves on Dijkstra by adding a heuristic estimate of cost-to-go, enabling the search to focus on states that appear closer to the goal. Its effectiveness depends on both the cost model and the heuristic function. For 4-connected grids with unit step costs, the Manhattan distance is admissible and consistent. For 8-connected grids, where diagonal moves are allowed with cost √2, the Euclidean distance provides a more accurate admissible heuristic. These variations highlight that A* is not a single method but a family of approaches, and the choice of heuristic directly influences runtime, node expansions, and overall efficiency while still preserving optimality.

D. *Theta* Algorithm*

Theta* is a variation of A* that allows any-angle shortcuts rather than restricting paths to grid edges. This reduces unnecessary turns and often produces shorter, more natural-looking paths, while maintaining efficiency.

E. *Jump Point Search (JOS)*

PS is an optimization for uniform-cost grids that skips over symmetric intermediate states. By "jumping" directly to critical nodes, JPS dramatically reduces the number of expansions while preserving optimality.

F. *Beyond Classical Grid Search*

Although the immediate scope of this project is restricted to the five grid-based planners above, robotics often requires planning in environments that are partially known or dynamic. Incremental algorithms such as D* and D* Lite update solutions efficiently when obstacles are discovered, while reinforcement learning methods such as Dyna-Q maintain policy-based decision structures through interaction with the environment. These approaches are not implemented here, but the framework is designed to be flexible and could support them in future work.

Evaluation of all these algorithms typically considers metrics such as:

- Path length (or cost)
- Number of turns
- Runtime performance
- Nodes expanded
- Memory consumption of open/closed lists

III. METHDOLOGY

The goal of this project is not only to implement individual planning algorithms, but to develop a consistent and extensible benchmarking framework. This section describes the design choices, software environment, and evaluation setup.

A. *Software Environment*

The framework is implemented in **ROS 2 Humble** using **Python 3.10**. A custom ROS 2 package, *rbe550_grid_bench*, provides the benchmarking utilities and planner implementations. The build system is based on *colon*, and helper scripts (build.sh and run.sh) simplify compilation, environment setup, and experiment execution.

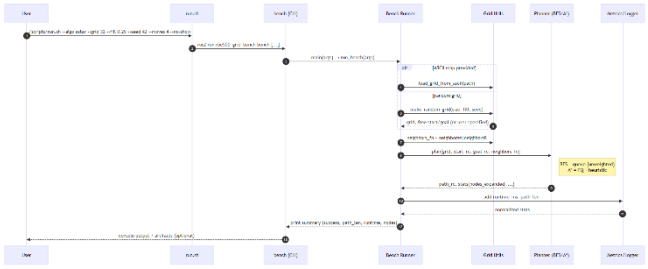

Figure I. Execution sequence from user command through CLI, grid construction, planner invocation, and metrics reporting.

B. *Grid Representation*

The environment is modeled as a 2D occupancy grid. Each cell is marked as either free or occupied, and planners operate on the induced grid graph. The framework supports:

- **Random grids**, generated with a target fill percentage of obstacles. A random seed is specified for reproducibility.
- **ASCII map files**, where obstacles and free spaces are explicitly encoded.

Start and goal cells are either randomly selected from free cells or specified by the user. To ensure validity, both start and goal are forced to free space.

C. *Planar Interface*

To maintain consistency, each algorithm is wrapped in a common Python interface:

```
def plan(grid, start, goal, neighbors_fn):
    return path, stats
```

Here, grid is a binary occupancy matrix, start and goal are given as (row, col) coordinates, and neighbors_fn specifies either 4-connected or 8-connected motion. Each algorithm returns a path as a list of coordinates and a stats dictionary containing relevant metrics (runtime, nodes expanded, path length, and memory usage where available).

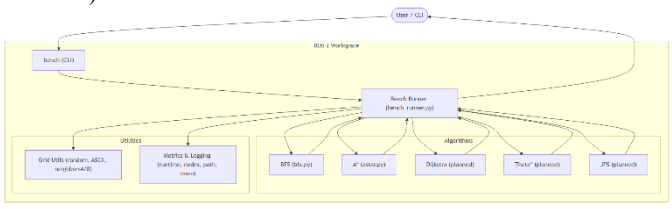

Figure II. High-level system architecture of the benchmarking framework, illustrating interactions between CLI, runner, planners, and metrics components.

D. *Implelemted Algorithms*

At this stage, Breadth-First Search (BFS) and A* are being integrated as baseline algorithms. BFS is implemented with a queue-based frontier and uniform step costs. A* uses a priority queue guided by admissible heuristics, with support for both Manhattan distance (appropriate for 4-connected grids with unit costs) and Euclidean distance (appropriate for 8-connected grids with diagonal moves costing √2). This allows benchmarking of multiple A* variants to study how heuristic selection influences efficiency and path quality. Additional algorithms—Dijkstra, Theta*, and Jump Point Search (JPS)—will be incorporated using the same interface to enable fair comparisons.

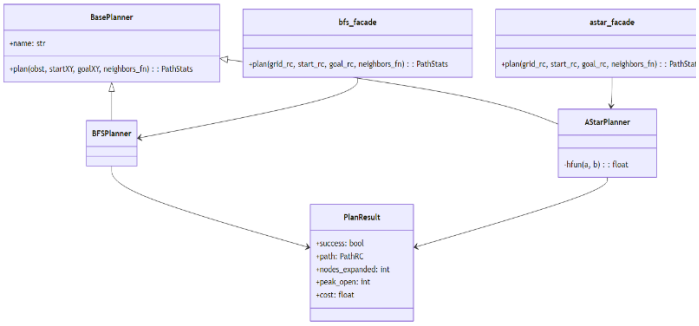

Figure III. Class diagram of algorithm modules, showing BFS and A planners inheriting from a common base planner interface. *

E. Benchmarking Harness

The benchmarking harness executes planners on specified grids and records standardized metrics:

- Runtime measured in milliseconds
- Nodes expanded during search
- Path length (number of steps)
- Number of turns (optional, for later integration)
- Memory usage of open and closed lists

Each run reports results in a structured format, printed to the console and optionally saved for later analysis. The harness ensures that all planners are evaluated under identical conditions. In particular, it supports varying the cost model (4-connected vs. 8-connected grids) and heuristic functions (Manhattan vs. Euclidean) so that differences between algorithmic approaches can be measured systematically.

F. Reproducibility and Scripts

Experiments are launched via run.sh, which sources the ROS 2 environment, builds the workspace, and invokes ros2 run rbe550_grid_bench bench with user-specified parameters. Command-line options allow configuration of grid size, fill percentage, seed, start/goal, algorithm selection, and connectivity. This design enables reproducible benchmarking across random and structured maps.

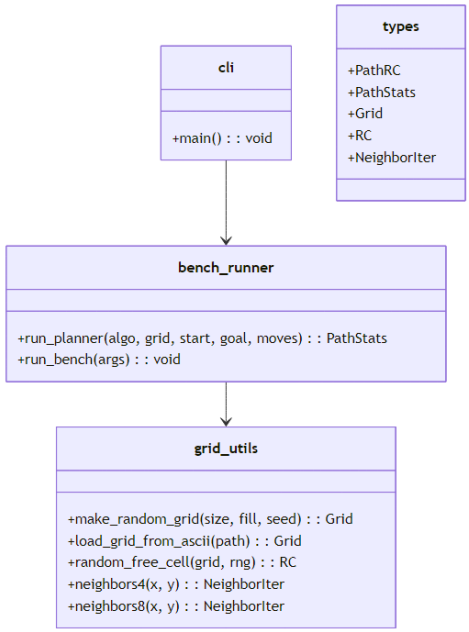

Figure IV. Class diagram of core data structures used in the framework, including planners, plan results, and grid utilities.

IV. Preliminary Results

V. Challenges and Next Steps

VI. Conclusion

References

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. SSC-4, no. 2, pp. 100–107, 1968.

[3] K. Daniel, A. Nash, S. Koenig, and A. Felner, “Theta*: Any-angle path planning on grids,” *J. Artificial Intelligence Research*, vol. 39, pp. 533–579, 2010.

[4] D. Harabor and A. Grastien, “Online graph pruning for pathfinding on grid maps,” in *Proc. AAAI*, 2011. (Jump Point Search)

[5] R. A. Dechter and J. Pearl, “Generalized best-first search strategies and the optimality of A*,” *J. ACM*, vol. 32, no. 3, pp. 505–536, 1985. (A* properties)

[6] S. Koenig and M. Likhachev, “D* Lite,” in *Proc. AAAI*, 2002. (optional background, incremental planning)

[7] S. M. LaValle, *Planning Algorithms*. Cambridge Univ. Press, 2006. (open-access textbook; grid search background)

[8] Open Robotics, “ROS 2 documentation (Humble),” 2022. [Online]. Available: docs.ros.org

[9] Mermaid Team, “Mermaid CLI,” 2024. [Online]. Available: mermaid.js.org

[10] PlantUML Team, “PlantUML,” 2024. [Online]. Available: plantuml.com